

CPEN 311 Lab 2 – Simple Datapath and Controller: Baccarat

1 Introduction

In this lab, you will become familiar with the software and hardware we will be using in the labs and use the hardware and software to implement a simple digital datapath.

We will be using two pieces of software for most of this course: Quartus Prime, which is produced by Intel, and ModelSim, which is made by Mentor Graphics. Several versions of ModelSim are available; we will be using one distributed with Quartus. You must download and install these programs (the free Lite version). You should have done this in the tutorial lab. We strongly recommend that you start working on labs early — it always takes longer than you think, especially if you are not very good at debugging. **This lab is relatively easy; future labs will be significantly harder and take longer.**

2 Baccarat

The result of this lab will be a Baccarat engine. Baccarat is a card game played in casinos around the world. Baccarat is for "high rollers" — James Bond is a fan of Baccarat (and, for some odd reason, martinis made with cheap vodka and tiny ice shards). Designing this Baccarat engine will help you understand how a state machine can construct and control a simple datapath; this is the foundation of all large digital circuits.

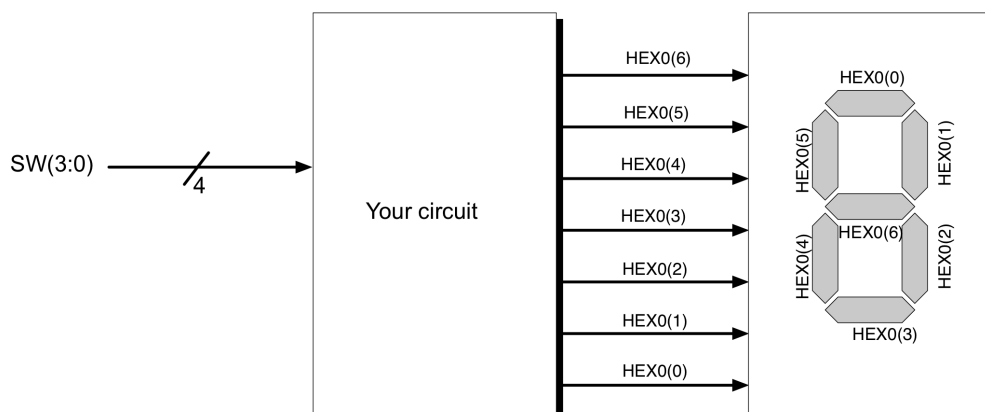
It may be tempting to write a software-like specification and hope the synthesis tools can synthesize the design into hardware. It is unlikely that this approach would be successful even for this simple design. In this lab, we will consider the circuit at the level of the hardware we want to build — to succeed in this course. You need to understand precisely what hardware you are making and exactly how this hardware works. This handout will take you through the steps necessary to construct the circuit, starting from the basic blocks and ending in a working circuit on your DE1-SoC board.

3 Phase 1: Getting your environment ready

The assumption is that you have completed the design flow tutorial. This will ensure you have the correct versions of the design tools and can use them to program your FPGA board. There are no deliverables for this task.

4 Phase 2: Warmup

To get you started quickly, in this Phase, you will create the simplest useful combinational circuit we can think of. It will also be a component in the Baccarat game you will create in Phase 3. You will design a combinational block with four inputs and seven outputs:



In this Phase, the four inputs will be connected to switches 3 to 0 on your board, and the seven outputs will be connected to a single seven-segment display on your board.

The circuit operates as follows. The four inputs represent an unsigned, 4-bit binary number with a value between 0 and 15. Each number between 1 and 13 represents a specific card value in a deck of cards: Ace=1, number cards are defined as themselves, Jack=11, Queen=12, and King=13. An input of 0 represents "no card," and input values of 14 and 15 will not be used; for this Phase, you should display a blank if they occur.

The output is a set of values that will display the value of the card on the seven-segment display:

SW(3 downto 0)	Output	SW(3 downto 0)	Output
0000 (blank)		1000	
0001 (Ace)		1001	
0010		1010	
0011		1011 (Jack)	
0100		1100 (Queen)	
0101		1101 (King)	
0110		1110 (not used)	
0111		1111 (not used)	

4.1 Task 1: A seven-segment LED driver

Add your code to implement the Verilog stub in the task1 folder. Create a project in Quartus, import the pin assignments, compile your design, download it to the board, and test it. Remember to import the correct pin assignment file from the settings folder for the board you use.

Write a module with unit tests for the LED driver in tb_card7seg.sv. Your testbench module will be named tb_card7seg and will have no ports. This means it will need to instantiate card7seg.sv, as well as and drive any clock(s) in the design using the Verilog delay syntax (#). Ensure your testbench covers all cases, even those for which the display is blank. You may use waveforms or text output (such as \$display, \$monitor, or \$strobe) to check whether your testbench works; we only care that it exercises the entire design under test.

Even though this module isn't very complex, we strongly recommend writing the testbench before implementing the block in Verilog. You should approach all modules this way; this will make your life much easier when working on more complex designs. It will force you to go through the specification carefully, ensure you understand it, and avoid leaking hidden assumptions from your design into your tests. Plus, it's satisfying to finish your code and see that it passes all your tests!

Be sure to test both the SystemVerilog RTL exhaustively code you write and the post-synthesis netlist Verilog file produced by Quartus (see the Tutorial).

5 Phase 3: Baccarat!

5.1 Task 2: Learn how to play Baccarat

The game of Baccarat is simple. There are various versions, but we will focus on the simplest, called Punto Banco, which is played in Las Vegas and Macau. The following text will describe the algorithm in sufficient detail for completing this lab; if you need clarification or more information on any point, there are numerous tutorials on the web (Google is your friend).

5.1.1 Rules

- Two cards are dealt to the player and the dealer (i.e., the banker) face up (first card to the player, second card to dealer, third card to the player, fourth card to the dealer).
- The score of each hand is computed as described under Score below.
- If the player's or banker's hand has a score of 8 or 9, the game is over (this is called a "natural"), and whoever has the higher score wins (if the scores are the same, it is a tie).
- Otherwise, if the player's score from their first two cards was 0 to 5:
 - The player gets a third card.
 - The banker may get a third card depending on the following rule:
 - * If the banker's score from the first two cards is 7, the banker does not take another card.
 - * If the banker's score from the first two cards is 6, the banker gets a third card if the face value of the player's third card is a 6 or 7.
 - * If the banker's score from the first two cards is 5, the banker gets a third card if the face value of the player's third card is 4, 5, 6, or 7.
 - * If the banker's score from the first two cards is 4, the banker gets a third card if the face value of the player's third card is 2, 3, 4, 5, 6, or 7.
 - * If the banker's score from the first two cards is 3, the banker gets a third card if the face value of the player's third card was anything but an 8.
 - * If the banker's score from the first two cards is 0, 1, or 2, the banker gets a third card.
- Otherwise, if the player's score from their first two cards was 6 or 7:
 - The player does not get a third card.
 - If the banker's score from their first two cards was 0 to 5:
 - * The banker gets a third card.
 - Otherwise, the banker does not get a third card.
- The game is over. Scores are computed as below. Whoever has the higher score wins, or if they are the same, it is a tie.

5.1.2 Score

The score of each hand is computed as follows:

- The value of each card in each hand is determined. Each Ace, 2, 3, 4, 5, 6, 7, 8, and 9 has a value equal to the face value (e.g. Ace has the value of 1, 2 is a value of 2, 3 has a value of 3, etc.). Tens, Jacks, Queens, and Kings have a value of 0.
- The score for each hand (which can contain up to three cards) is then computed by summing the values of each card in the hand, and the rightmost digit (in base 10) of the sum is the score of the hand. In other words, if Value1 to Value3 are the values of Card 1 to 3, then:

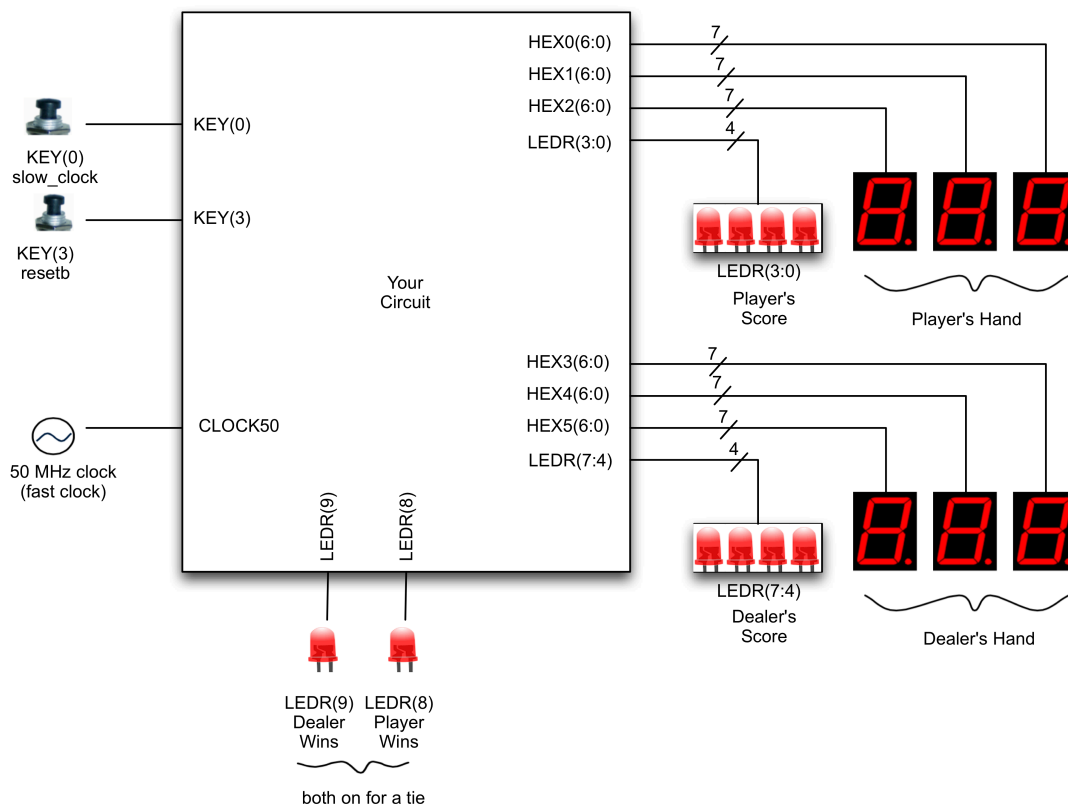
$$\text{Score of hand} = (\text{Value1} + \text{Value2} + \text{Value3}) \bmod 10$$

If the hand has only two cards, then Value3 is 0. You should see that the score of a hand is always in the range [0,9].

Interestingly, in this game version, all moves are automatic (the player does not have to make any decisions!). The version played in Monte Carlo is slightly different in that a player can choose whether or not to take a third card. We will not consider that here.

5.2 Task 3: Understand the overall functionality your circuit should implement

First, consider the behaviour of the Baccarat circuit from the user's point of view. As shown in the figure below, the circuit is connected to two input keys a 50MHz clock, and the output of the circuit drives six seven-segment LEDs and ten lights.



The game starts by asserting the reset signal (KEY3), which is **active-low**. The user can then step through each step of the game (deal one card to the player, one to the dealer, etc.) by pressing KEY0 (this will be referred to as **slow_clock** in this document). The exact sequence of states will be described below. As the cards are dealt, the player's hand is shown on HEX0 to HEX2 (one hex digit per card — remember each hand can contain up to three cards) and the dealer's hand is shown on HEX3 to HEX5. The current score of the player's hand will be shown on lights LEGR3 to LEGR0 (recall that the score of a hand is always in the range [0,9] and can be represented using four bits), and the current score of the dealer's hand will be shown on LEGR7 to LEGR4. We use lights to display the binary version of the score since the DE1-SoC only has six hex digits.

There is also a 50MHz clock input, used solely for clocking the dealcard block, which deals with a random card. This will be described further in a subsequent task.

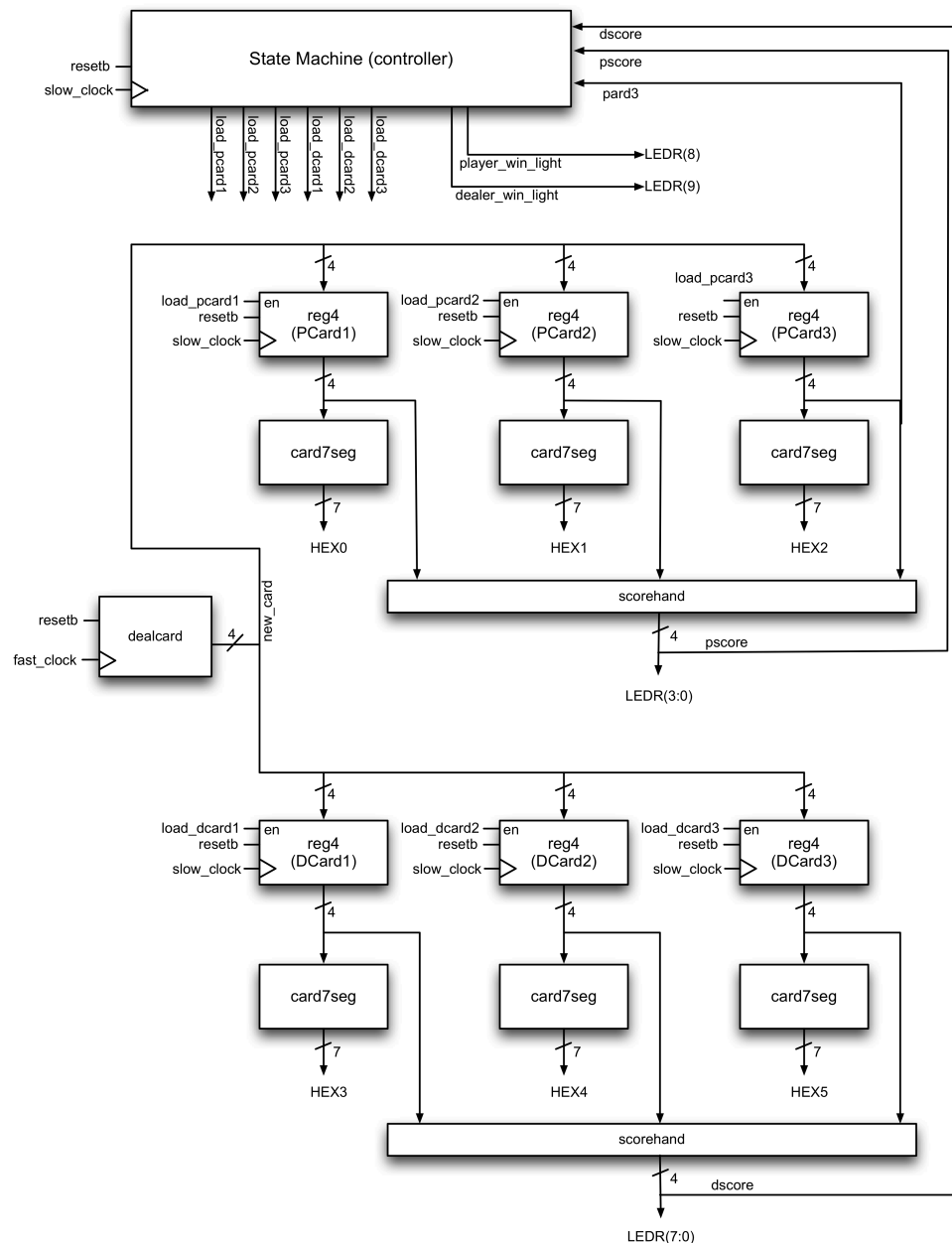
At the end of the game, red lights 8 and 9 will indicate the winner: if the player wins, light LEDR(8) goes high. (In your implementation, you may delay this until KEY0 has been pressed one more time after the winning card has been dealt). If the dealer wins, light LEDR(9) goes high. If it is a tie, both LEDR(8) and LEDR(9) go high. The system then does nothing until the user presses reset, sending it back to the first state to deal with another hand.

Notice that the user does not need to do anything besides cycling through the states using KEY0 (the slow clock). This is consistent with the description of the game above.

5.3 Task 4: Understand the components of your circuit

The circuit consists of two parts: a state machine and a datapath. The datapath does all the “heavy lifting” (in this case, keeping track of each hand and computing the score for each hand), and the state machine controls the datapath (in this case, telling the datapath when to load a new card into either the player's or dealer's hand). The overall block diagram is shown below.

First, consider the datapath, which consists of everything except the statemachine block in the block diagram. There are several subcircuits here, and each will be described below:



5.3.1 dealcard

To deal random cards, we need a random number generator. Random numbers are difficult to generate in hardware (can you suggest why?). We will use a few simple tricks. First, assume we are dealing from an infinite deck, so it is equally likely to generate any card, regardless of which cards have been given out (casinos try to approximate this approach using multiple decks). Second, assume that when the player presses the “next step” key, an unpredictable amount of time has passed, representing a random delay. The subcircuit is described in `dealcard.sv` during this random delay interval.

We have written `dealcard.sv` for you to save you time. This block has two inputs (the fast clock and a reset) and one output (the card being dealt, represented by a number from 1 to 13 as described above). This circuit is essentially a counter. Be sure you understand it before moving on.

The design of `dealcard` raises an interesting point. Because you have two clocks (one from KEY0 driving the modules that use `dealcard` and the other from the 50MHz clock driving `dealcard` itself), you may rarely observe that the card dealt is not in the range 1..13. This is because the two clocks are asynchronous — i.e., you might push KEY0 at just the right moment to sample the `dealcard` output when it is unstable. Can you reproduce this behaviour when testing in the FPGA?

You do not need to worry about the asynchronous clocks for this lab other than noticing when it happens. We will learn how to safely transmit signals among multiple clock domains and deal with asynchronous circuits later in the course.

5.3.2 reg4

Each card in each hand is stored in a **reg4** block, which is a 4-bit wide register (set of four D-flip-flops). The upper three **reg4** blocks store the player's hand, and the lower three **reg4** blocks store the dealer's hand (recall each hand can have up to three cards). Each card is stored as a number from 1 to 13 (Ace=1, number cards are represented as themselves, Jack=11, Queen=12, King=13). We will not store the suit information for each card (the suit of a card does not matter in Baccarat). If a position in the hand does not have a card, we store a 0 to represent "no card." For example, if the player's hand consists of a 5 and a Jack (and no third card), **PCard1** would contain the number 5, **PCard2** would contain the number 11, and **PCard3** would contain the number 0 (no card). Note that since there are 14 different possible values (including 0), four bits in each register are sufficient.

Each register is clocked using **slow_clock** (which is connected to KEY0 and toggled by the user). On each rising clock edge of **slow_clock**, if the enable signal (for **PCard1** the enable signal is called **load_pcard1**) is high, the value from **dealcard** is loaded into the register. The register also contains an active-low reset signal (which is connected to KEY3); when this is low, the value in the register goes to 0.

5.3.3 card7seg

This is the block you wrote in Phase 2. As you recall, this is a simple combinational circuit with a single 4-bit input (the value of a card encoded as above) and 7 outputs that drive a HEX according to the following pattern:

- The value 0 is "no card" and should be displayed as a blank (all HEX segments off).
- 1 is displayed as "A", 10 is displayed as "0", Jack as "J", Queen as "q", and King as "H".
- 2 through 9 are displayed as themselves, ensuring the numeral 9 appears differently than "q".

You can use the block from Phase 2 directly; the inputs now come from registers rather than switches.

5.3.4 scorehand

This is a simple combinational circuit that takes the value of three cards and computes the score of that hand. Recall that the score of a hand is computed as follows:

- The value of each of the three cards in each hand is determined. Each Ace, 2, 3, 4, 5, 6, 7, 8, and 9 has a value equal to the face value (e.g. Ace has a value of 1, 2 is a value of 2, 3 has a value of 3, etc.). Tens, Jacks, Queens, and Kings have a value of 0. If fewer than three cards are in the hand, the missing positions are 0.
- The values of the cards in each hand are summed, and the rightmost digit (in base 10) of the sum is the score of the hand. In other words, if Value1 is the value of the first card, Value2 is the value of the second card, and Value3 is the value of the third card, then:

$$\text{Score of hand} = (\text{Value1} + \text{Value2} + \text{Value3}) \bmod 10$$

You should be able to see that a hand can have a score in the range [0,9]. Thus, 4 bits are sufficient for the output of this block.

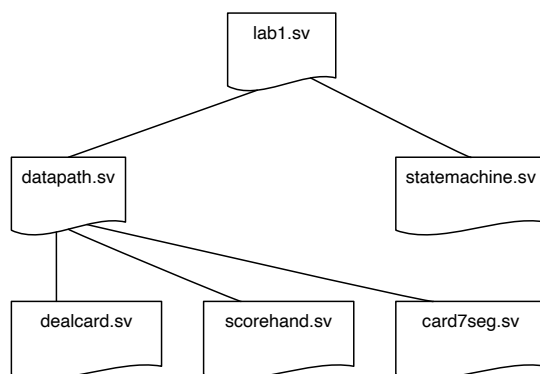
5.3.5 statemachine

The state machine is the "brain" of our circuit. It has an active-low reset (called **resetb**) and is clocked by **slow_clock** (which is connected to KEY0). On each rising edge of **slow_clock**, the state machine advances one step through the algorithm and asserts the appropriate control signals at each step. In this circuit, the control signals that the state machine controls are **load_pcard1**, **load_pcard2**, ... **load_dcard3**. When it is time to deal the first card to the player, the state machine asserts **load_pcard1**, which, as was described above, causes the first Reg4 block to load in a card (from the output of the **dealcard** block). During the cycle in which **load_pcard1** is 1, all other **load_pcard** and **load_dcard** signals are 0, so that no other positions in either hand are updated. As the algorithm progresses, the state machine will generate the other control signals to be asserted at the appropriate times.

As should be evident from the earlier discussion, the card drawing pattern depends on the dealer's score and the player's score (these are used to determine whether a third card is necessary) as well as the player's third card (this is used to determine whether the dealer should receive a third card, as described in the rules). Therefore, pcard3, pscore, and dscore are inputs to the state machine.

5.4 Task 5: Baccarat code and testbenches

In this task, you will create the testbenches and implement the design in Verilog. Your design must follow the hierarchical design approach shown below:



To get you started, stubs for each file are in the **task5** folder. Be sure to start with these so that your interfaces for each module are correct (do not modify the interfaces). The **reg4** block is not shown in the diagram; you can either create a new module to describe a four-bit register or write it directly into **datapath.sv** (your choice, either will work). To help you, we are giving you **dealcard.sv** and **task5.sv**.

Start by writing unit tests for all your modules (you don't need to test **dealcard**, **card7seg**, or **reg4**). Each **tb_*.sv** file should test the corresponding module by providing inputs to the module's ports, examining the outputs, and testing all of the code in the module. This also applies to the testbench for the toplevel module **task5**, which should only interface with the **task5** module and should not include the unit testbenches.

Be sure to test **both** the SystemVerilog RTL exhaustively code you write and the post-synthesis netlist Verilog file produced by Quartus (see the Tutorial).

The hardest part of this lab is getting the state machine right. We strongly urge you to draw (on paper) a bubble diagram showing the states, the transitions, and the outputs of each transition. You are strongly urged to make sure you have your state machine bubble diagram done by the end of your Working Week lab. If you are unclear on how to do this, be sure to **discuss it with your TA during the lab period or office hours**. When drawing your diagram for your state machine, make sure that you cover all of the possible input conditions.

5.5 Task 6: Verify your design on a DE1-SoC board

Test your design by downloading it to the board and demo the working circuit to the TA. Remember that you should treat your TA as a client, and it is up to you to devise an appropriate demo that shows that your design works. The TA will ask questions to gauge your understanding of the lab.

Remember to submit your updated folder with updated .sv files to Canvas before the deadline. You will need to submit a zip file. If you forget this, you will receive 0 marks for the lab. If your design works, you do not need to demo the datapath and state machine separately, just the entire design. However, if you cannot get a working design, you should prepare to demo as many subunits on the DE1-SoC board as possible to the TA.

6 Deliverables and Evaluation

6.1 Using Canvas

You should submit **only** source files (in this lab, **.sv** files), and, optionally, the post-synthesis netlist files generated by Quartus (the **.vo** files). You do **not** need to submit the bitfile for programming the FPGA (e.g., **.sof** files), waveform dumps, temporary files, etc. In particular, be careful not to submit any **extra .sv** files that your design does not use; our testing environment will process your submission as described below, and extra files that do not compile will cause you to fail the tests.

Any template files we give you (e.g., **card7seg.sv**) should be directly modified and submitted using **the same filename**. **Do not share your code or look at other codes. We will run academic integrity checks in the month of December for all the labs. If you are deemed to have cheated, we will follow the academic integrity procedures mandated by the department and UBC to the fullest extent.**

6.2 Submission Rules for all students

Please follow these rules strictly.

- You must not rename any files we have provided.
- You must not **add** any files that contain unused Verilog code.
- Your testbench files must begin with **tb_** and **correspond to design file names** (e.g., **tb_scorecard.sv** for **scorecard.sv**).
- You must **not have multiple copies of the same module** in separate submitted source files in the same task folder. This will cause the compiler to fail because of duplicate module definitions.
- Your modules must **not rely on files from another folder** (e.g., you will need a separate copy of **card7seg.sv** in the **task5** folder, as well as separate copies of all relevant files in **task7**).
- You must **not alter the module declarations, port lists, etc.**, in the provided skeleton files.
- You must **not rename any modules, ports, or signals** in the provided skeleton files.
- You must **not alter the width or polarity of any signal** in the skeleton files (e.g., **resetb** must remain active-low).
- You must submit the **.vo** files you synthesized.
- You must submit a zip file of the lab folder as the final submission.

6.3 Submitting with a partner

You do not need a partner, but you may have one if you like.

- If you have a partner, **both you and your partner need to submit the exact same zip file** using your respective Canvas logins. If only one of you submit, you both will get 0 marks.
- **Both partners must write the name of their respective partner as a comment on their submission. Failing to do so will result in 0 marks.**
- Your partner must be **from the same lab section** and **must be present during the grading process**.
- **You and your partner must arrive at the lab right at the beginning of the grading lab** and write your names on a sheet that will be provided by the TA. **If you or your partner arrive late, both you and your partner will get 0 marks.**
- The TA has finite hours, and we cannot wait for team members to arrive at arbitrary times, regardless of whether it is your fault or not. **So please choose your partner (if you want one) very carefully.**

7 Post-synthesis simulation

Be sure to test both the SystemVerilog RTL exhaustively code you write and the post-synthesis netlist Verilog file produced by Quartus (see the Tutorial). It is entirely possible to write “unsynthesizable” RTL that works “in the simulation,” but either fails to synthesize or synthesizes into something that behaves differently.

Marking: Total 12 Marks

The marks are assigned in the following manner. 70% of your grade will be from the in-person demo for each task. The remaining 30% of your grade will be from answering the questions about your code by your TA.

- **Task 2: 3 Marks**

Deliverables should be in the folder `task1` (yes, that says `task1`):

- Modified `card7seg.sv`
- Modified `tb_card7seg.sv`
- Any other modified/added source or test files for your design.

- **Task 5: 9 Marks**

Deliverables should be in the folder `task5`:

- Modified `card7seg.sv`
- Modified `statemachine.sv`
- Modified `scorehand.sv`
- Modified `datapath.sv`
- Modified `tb_statemachine.sv`
- Modified `tb_scorehand.sv`
- Modified `tb_datapath.sv`
- Modified `tb_task5.sv`
- Any other modified/added source or test files for your design.

The toplevel module of your design must be named `task5`.